Modelo de programación, memoria y datos

Sistemas Operativos · Semana 2

Cap. 1: Introducción (temas 1.4 – 1.6)

Rubén Rumipamba

Escuela Politécnica Nacional

Agenda de la semana

SESIÓN 1 · 2 H · TEMAS 1.4 Y 1.6

Modelo de programación y datos

- Registros, ISA, CISC/RISC
- Modos usuario/núcleo
- Pila y llamadas
- Complemento a 2, endianness
- Reto: volcado hexadecimal

SESIÓN 2 · 2 H · TEMA 1.5

Jerarquía de memoria

- Brecha CPU–memoria
- Localidad (demo en C)
- Caché: líneas y campos
- T medio y ejercicios
- Laboratorio con el simulador

SESIÓN 3 · 1 H · REPASO

Consolidación

- Quiz de 5 preguntas
- Caso: el cocinero que cruza la cocina un millón de veces
- Deber 2 (en parejas)
- Temas para revisión autónoma

Objetivos de la semana

1	Describir el modelo de programación de una CPU (registros, modos, pila) y relacionarlo con el SO.	Sesión 1	RA1
2	Interpretar enteros, texto y reales en binario/hex, incluyendo desbordamiento y endianness.	Sesión 1	RA1
3	Explicar la jerarquía de memoria y estimar el tiempo medio de acceso de una caché.	Sesión 2	RA1
4	Automatizar en parejas un reporte de la memoria del equipo con Bash y PowerShell.	Sesión 3	RA2 · RA3

Pregunta de la semana: ¿qué puede ver y tocar un programa, y por qué pasa tanto tiempo esperando a la memoria?

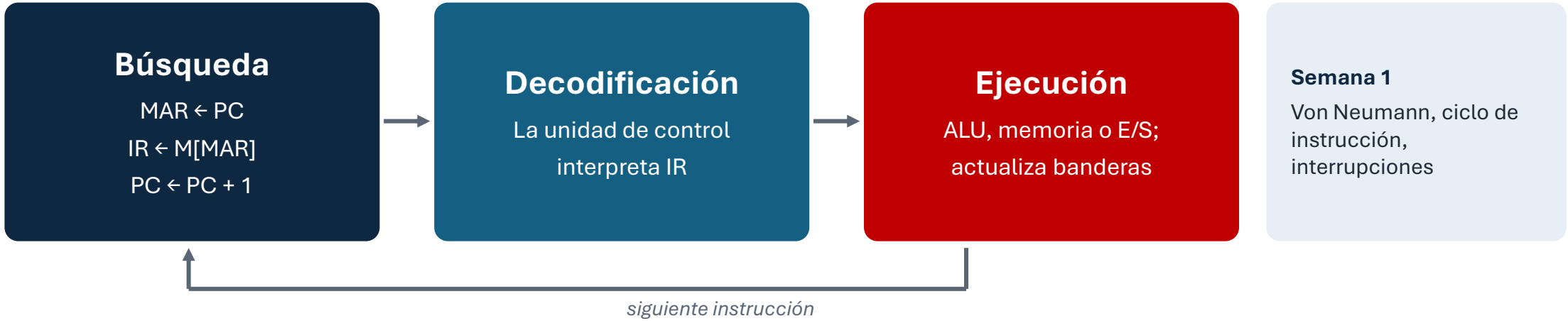


Sesión 1 · Lo que ve el programador

Temas 1.4 y 1.6 · registros, modos, pila y representación de datos · 1 h 45 min

Repaso: la CPU repite un ciclo

El cocinero sigue su receta, paso a paso



Semana 1

Von Neumann, ciclo de instrucción, interrupciones

Pregunta de hoy: el cocinero (un núcleo de la CPU) lee la receta (el programa) paso a paso. ¿Qué tiene a su alcance mientras cocina? Esa «vista» de la CPU se llama modelo de programación.

¿Qué es el modelo de programación?

¿Qué tiene el cocinero a su alcance?

1 · En la mano

Pocos utensilios, los que usa en este instante. Tomarlos es inmediato.

2 · Su repertorio

Las técnicas que sabe hacer: picar, mezclar, hornear. Ni una más.

3 · Su puesto y zonas con llave

Trabaja libre en su puesto; algunas zonas solo se abren con el gafete del administrador.

4 · Notas y timbres

Lo interrumpen; pega notas de «dónde me quedé» para volver.

De la cocina a la CPU

En la cocina	En la computadora
Lo que tiene en la mano	**Registros**
Las técnicas que sabe hacer	**Juego de instrucciones (ISA)**
Su puesto / zonas con llave (gafete)	**Modo usuario / modo núcleo**
Timbres y notas	**Interrupciones y pila**

Idea clave

El modelo de programación es lo que la CPU deja **ver y usar** a un programa: el contrato entre hardware y software.

La caché o el pipeline existen, pero la receta no los nombra: el cocinero los usa por su cuenta.

ISA: Instruction Set Architecture

Carretero et al. (2007), Sistemas operativos: una visión aplicada, cap. 1 · Stallings (2006), OAC, cap. 10–12.

De la máquina de Stallings a una CPU real

Los utensilios de la semana 1, con su nombre real

Semana 1 (Stallings)	En x86-64	¿Lo ve el programa?
PC · contador de programa	`RIP`	Sí, indirectamente (saltos, CALL, RET)
PSW · palabra de estado	`RFLAGS`	Sí (saltos condicionales)
AC · acumulador	`RAX (uno de 16 generales)`	Sí
SP · puntero de pila	`RSP`	Sí
IR, MAR, MBR	`sin nombre visible`	No: son internos de la CPU

Nombre de libro

`PC, PSW, AC`

Nombre en Intel

`RIP, RFLAGS, RAX`

Idea clave

Misma función, otro nombre. Y el programa solo ve **una parte** de los registros de la semana 1.

Stallings (2006), OAC, cap. 12 «Estructura y funcionamiento del procesador» · Intel (2024), SDM vol. 1, cap. 3.

¿Por qué tantos nombres?

8086 · 1978 — 16 bits

IP	FLAGS	AX

Nombres originales

→

80386 · 1985 — 32 bits

EIP	EFLAGS	EAX

E = extended

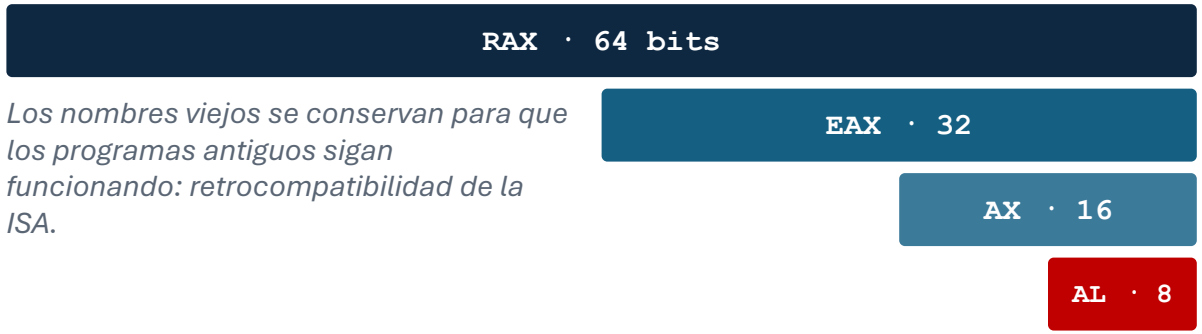
→

AMD64 · 2003 — 64 bits

RIP	RFLAGS	RAX

R = register; se añaden R8–R15

Un registro, cuatro nombres



Los nombres viejos se conservan para que los programas antiguos sigan funcionando: retrocompatibilidad de la ISA.

Los 16 generales (hoy usaremos los resaltados)

RAX	RBX	RCX	RDX	RSI	RDI	**RBP**	**RSP**
R8	R9	R10	R11	R12	R13	R14	R15

RIP **RFLAGS** *especiales*

Banderas de RFLAGS que usaremos

CF	ZF	SF	IF	OF
acarreo	cero	signo	interr.	desbord.

Intel (2024), SDM vol. 1, cap. 3 «Basic Execution Environment» · AMD (2023), AMD64 Architecture Programmer's Manual, vol. 1.

Dos ISA que usan a diario: x86-64 y ARM64

Por qué la receta de tu laptop no sirve en tu celular

	x86-64 (Intel, AMD)	ARM64 / AArch64
Registros generales	16 de 64 bits	31 de 64 bits (X0–X30)
Longitud de instrucción	Variable: 1 a 15 bytes	Fija: 4 bytes
Acceso a memoria	Muchas instrucciones operan directo con memoria	Solo LDR/STR tocan memoria (load/store)
Dónde está	PC de escritorio, servidores	Celulares, Apple M, Raspberry Pi, nube

Para el SO: Linux y Windows se compilan para ambas ISA; el núcleo tiene una capa por arquitectura (en Linux, arch/x86 y arch/arm64).

Arm Ltd. (2024), Arm Architecture Reference Manual for A-profile · Intel (2024), SDM vol. 1 · Hennessy y Patterson (2019), apéndice K.

CISC frente a RISC: dos filosofías

Recetas de pasos grandes o de pasos pequeños

CISC · x86-64

- Instrucciones complejas y de longitud variable
- Una instrucción puede leer memoria y operar
- Código más compacto; decodificación más compleja

ensamblador

```
; a = a + b   (b en memoria)
add  eax, DWORD PTR [rbx]
```

RISC · ARM64

- Instrucciones simples de 4 bytes
- Primero cargar en registro, luego operar
- Más instrucciones, pero fáciles de segmentar

ensamblador

```
// a = a + b   (b en memoria)
ldr  w1, [x1]
add  w0, w0, w1
```

Stallings (2006), OAC, cap. 13 «Computadores de repertorio reducido de instrucciones» · Patterson y Hennessy (2017), cap. 2.

Modo usuario y modo núcleo

Zonas donde solo entra el administrador

1 · Su puesto

El cocinero prepara el plato en su puesto, con sus utensilios.

2 · Puertas con llave

Llave del gas, caja fuerte, bodega: así como está, no puede entrar.

3 · El gafete

Con el gafete del administrador sí entra, y sigue sus instrucciones.

4 · El administrador

Él decide cuándo y para qué se usa el gafete. Ningún plato lo toma solo.

De la cocina a la CPU

En la cocina	En la computadora
Cocinero preparando un plato	**Modo usuario (anillo 3)**
Cocinero con el gafete puesto	**Modo núcleo (anillo 0): ejecuta código del SO**
El gafete	**Bit de modo de la CPU (CPL)**
Lo que solo se hace con gafete	**Instrucciones privilegiadas**

Idea clave

Sin los dos modos, cualquier programa podría **apagar las interrupciones** y quedarse con la CPU.

El SO no tiene CPU propia: usa los mismos núcleos, con el gafete puesto.

Intel (2024), SDM vol. 3A, cap. 5 «Protection» · Tanenbaum y Bos (2022), Modern Operating Systems, sec. 1.3.1.

Anillos de protección en x86

Las zonas de la cocina, vistas en la CPU



Linux y Windows usan solo los anillos 0 y 3.

Un bit de modo: el gafete

El nivel actual de privilegio (CPL) viaja con el estado de la CPU. En modo usuario el programa no puede tocar el hardware directamente.

Instrucciones privilegiadas (solo anillo 0)

`HLT`	detener la CPU
`CLI / STI`	desactivar o activar interrupciones
`IN / OUT`	acceso directo a puertos de E/S
`MOV CR3, …`	*para curiosos: cambiar la tabla de páginas (cap. 6)*

Intel (2024), SDM vol. 3A, cap. 5 «Protection» · Tanenbaum y Bos (2022), Modern Operating Systems, sec. 1.3.1.

¿Cómo se pasa al modo núcleo?

Tres formas de llamar al administrador: timbre, accidente, ventanilla

1 · Timbre

Alguien de fuera toca el timbre. Llega en cualquier momento.

2 · Accidente

El cocinero se quema o intenta abrir la caja fuerte. Lo causa su propio paso.

3 · Ventanilla

El plato necesita harina de la bodega: el cocinero hace un pedido.

4 · Una sola puerta

Siempre por la misma puerta, con el gafete. Luego el cocinero vuelve al plato.

De la cocina a la CPU

En la cocina	En la computadora
Timbre	**Interrupción (reloj, disco, teclado)**
Accidente	**Excepción (división por 0, instrucción privilegiada)**
Ventanilla	**Llamada al sistema: SYSCALL · semana 4**
La puerta fija	**Tabla de vectores que el SO configura al arrancar**

Idea clave

El programa nunca «salta» al núcleo por su cuenta: entra por la puerta que el SO preparó.

Y al volver… ¿cómo sabe dónde se quedó?

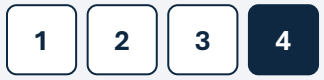
Stallings (2018), Operating Systems, sec. 1.4 y 2.3 · Carretero et al. (2007), cap. 1 «Interrupciones y llamadas al sistema».

La pila: retomar lo que se interrumpió

¿Cómo sabe el cocinero dónde se quedó?

1 · Sigue la receta

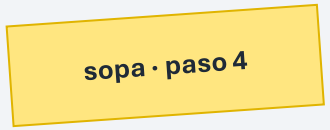
Va en el paso 4 de la receta de la sopa.



receta de la sopa

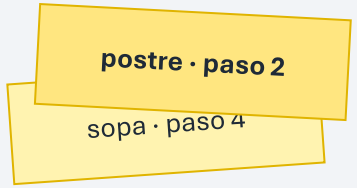
2 · Lo llaman

Antes de ir, pega una nota: «iba en el paso 4».



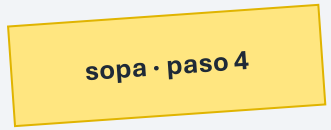
3 · Lo vuelven a llamar

Pega otra nota encima: «postre, paso 2».



4 · Vuelve

Quita la nota de arriba y sigue. Luego la siguiente.



De la cocina a la CPU

En la cocina	En la computadora
La nota «iba en el paso 4»	**Dirección de retorno (el PC guardado)**
El montón de notas	**La pila (zona de memoria)**
La nota de arriba	**La cima, señalada por RSP**
Quitar la nota de arriba y seguir	**RET (IRET si fue una interrupción)**

Idea clave

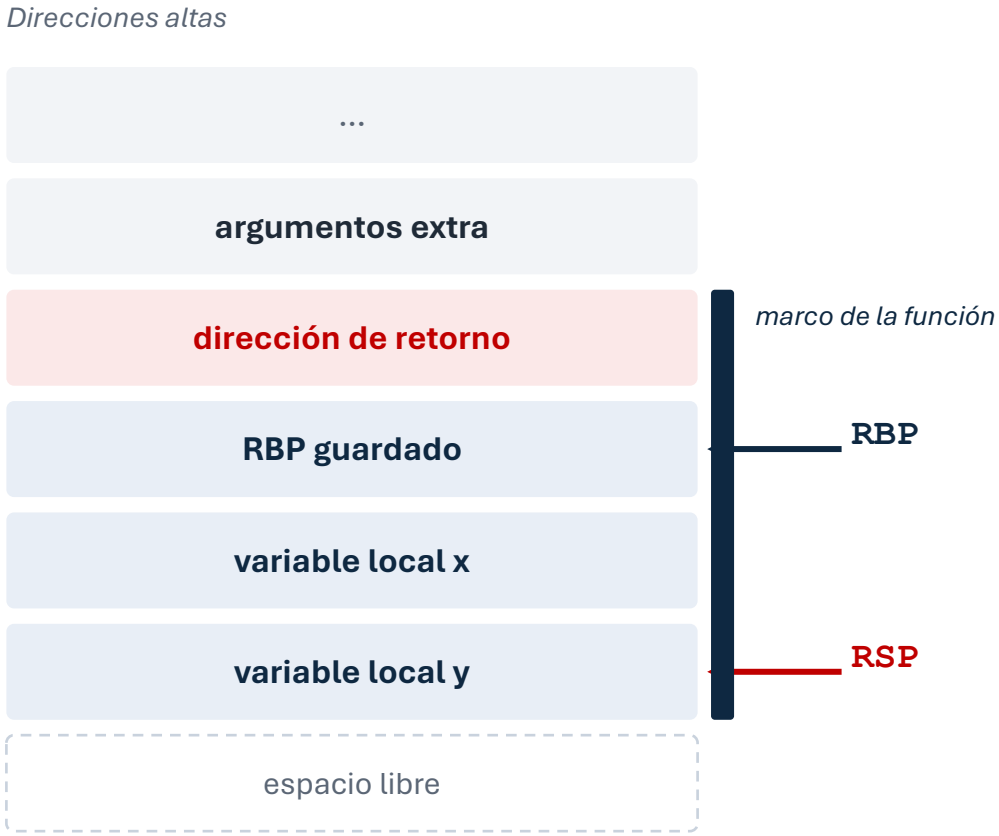
Lo último que se interrumpe es lo primero que se retoma: una **pila (LIFO)**.

Una función que llama a otra usa el mismo truco.

Stallings (2006), OAC, apéndice 10A «Pilas» · Bryant y O'Hallaron (2016), CS:APP, sec. 3.7.

La pila en memoria

El montón de notas, visto en la memoria



PUSH x	`RSP ← RSP − 8;  M[RSP] ← x`
POP x	`x ← M[RSP];  RSP ← RSP + 8`
CALL f	`PUSH dirección de retorno;  RIP ← f`
RET	`POP RIP: vuelve a quien llamó`

Idea clave

Cada función tiene su marco; RSP señala siempre la nota de arriba.

Bryant y O'Hallaron (2016), CS:APP, sec. 3.7 «Procedures» · Stallings (2006), OAC, apéndice 10A «Pilas».

PUSH, CALL y RET paso a paso

Pegar y quitar notas, instrucción por instrucción

programa.s

```
0x400:  push  rax
0x401:  call  0x500
0x406:  add   rbx, 1

0x500:  ...     ; f
0x5A0:  ret
```

Estado inicial

```
RIP = 0x400
RSP = 0x7FF0
RAX = 0x2A
```

1 · push rax

0x7FE8	0x2A

RIP = **0x401**
RSP = **0x7FE8**

2 · call 0x500

0x7FE8	0x2A
0x7FE0	0x406

RIP = **0x500**
RSP = **0x7FE0**

3 · ret

0x7FE8	0x2A
0x7FE0	0x406

RIP = **0x406**
RSP = **0x7FE8**

Tras RET la celda 0x7FE0 conserva 0x406 (borde punteado), pero ya está «fuera» de la pila: RSP volvió a 0x7FE8.

Demo: de C a ensamblador

La receta traducida al idioma del cocinero

suma.c

```c
int suma(int a, int b) {
    int r = a + b;
    return r;
}
```

Qué observar

- Los argumentos llegan en EDI y ESI
- El resultado sale en EAX
- Con -O2 todo queda en `lea eax,[rdi+rsi]` + ret

x86-64 gcc 14 · -O0

```
suma:
  push   rbp              ; guarda RBP
  mov    rbp, rsp         ; nuevo marco
  mov    DWORD PTR [rbp-20], edi  ; a
  mov    DWORD PTR [rbp-24], esi  ; b
  mov    edx, DWORD PTR [rbp-20]
  mov    eax, DWORD PTR [rbp-24]
  add    eax, edx         ; actualiza banderas
  mov    DWORD PTR [rbp-4], eax   ; r
  mov    eax, DWORD PTR [rbp-4]
  pop    rbp              ; restaura
  ret
```

Compiler Explorer, godbolt.org · Convención de llamada System V AMD64 ABI (Linux) · Bryant y O'Hallaron (2016), sec. 3.7.

Bits, bytes y hexadecimal

Un dígito hexadecimal = 4 bits

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Ejemplo: un byte

1	0	0	1	1	1	0	0

1001 → 9 1100 → C

0x9C = $1001\ 1100_2$ = 156_{10}

Prefijos: 0x (C, Bash, PowerShell) · h (ensamblador Intel)

Por qué hex: es la forma compacta de leer bits. Las direcciones, los volcados de memoria y los permisos de archivos se muestran en hex u octal.

Bryant y O'Hallaron (2016), CS:APP, sec. 2.1 «Information Storage».

Enteros con signo: complemento a 2

100
`0110 0100`
valor positivo

→

invertir
`1001 1011`
cambiar 0 ↔ 1

→

+ 1
`1001 1100`
= 0x9C = −100

Bits (8)	Sin signo	Con signo (C2)
`0000 0000`	0	0
`0111 1111`	127	127 (máximo)
`1000 0000`	128	−128 (mínimo)
`1001 1100`	156	−100
`1111 1111`	255	−1

Reglas

- El bit más alto indica el signo (1 = negativo).
- Rango con n bits:
  $$-2^{n-1} \ \ldots \ 2^{n-1} - 1$$
- La suma es la misma para con y sin signo: la ALU no necesita saber cuál es.

Bryant y O'Hallaron (2016), CS:APP, sec. 2.2 «Integer Representations» · Stallings (2006), OAC, cap. 9.

Desbordamiento: cuando el resultado no cabe

Ejemplo en 8 bits: 120 + 10

	Bits	Sin signo	Con signo
A	`0111 1000`	120	120
B	`0000 1010`	10	10
A + B	`1000 0010`	130	−126 ✗

1. Sin signo: 130 cabe (≤ 255) → **`CF = 0`**
2. Con signo: dos positivos dieron un negativo → **`OF = 1`**
3. **`ZF = 0`**
4. **`SF = 1`**

`¡El Software (programa) gestiona el error!`

CF — **Acarreo**
Sale un 1 del bit más alto. Error si los números son sin signo.

OF — **Desbordamiento**
Operandos del mismo signo y resultado de signo contrario.

ZF — **Cero**
Todos los bits del resultado son 0.

SF — **Signo**
Copia del bit más alto del resultado.

En C: el desbordamiento de int con signo es comportamiento indefinido; muchos fallos de seguridad empiezan así.

Intel (2024), SDM vol. 1, sec. 3.4.3 «EFLAGS Register» · Bryant y O'Hallaron (2016), sec. 2.3.

Endianness: el orden de los bytes en memoria

Guardar el entero de 32 bits **`0x12345678`** a partir de la dirección 0x1000

	0x1000	0x1001	0x1002	0x1003
Little-endian x86-64, ARM (habitual)	78	56	34	12
Big-endian orden de red (TCP/IP)	12	34	56	78

- byte menos significativo
- byte más significativo

Cuándo importa

Al leer un volcado de memoria, un archivo binario o un paquete de red generado por otra máquina.

En la práctica

Las funciones htonl y ntohl convierten entre el orden del equipo y el orden de red.

Cohen, D. (1981), «On Holy Wars and a Plea for Peace», IEEE Computer 14(10) · Bryant y O'Hallaron (2016), sec. 2.1.3.

Demo: mirar los bytes de un archivo

bash · Linux

```
# Caso 1: Bytes en formato UTF-8
$ printf 'Año\n' > t.txt
$ xxd t.txt
00000000: 41c3 b16f 0a       A..o.

# Caso 2: un entero de 32 bits con signo
$ python3 -c "import struct;
  open('n.bin','wb').write(
  struct.pack('<i', -100))"
$ xxd n.bin
00000000: 9cff ffff          ....
```

PowerShell · Windows

```
# Caso 1: Bytes en formato UTF-8
PS> 'Año' | Set-Content t.txt `
      -Encoding utf8NoBOM
PS> Format-Hex t.txt

  # (salida abreviada)
  Offset Bytes                 Ascii
  000000 41 C3 B1 6F 0D 0A     A..o..

# Windows termina las líneas con
# 0D 0A (CR LF); Linux solo con 0A
```

41
«A» en ASCII: 1 byte

C3 B1
«ñ» en UTF-8: 2 bytes

9C FF FF FF
−100 en 32 bits, little-endian

Comandos: xxd (paquete vim-common/xxd) · Format-Hex (PowerShell 7) · RFC 3629, UTF-8.

Ejercicio resuelto · Sumas de 8 bits y banderas

Calcula el resultado de 8 bits y el valor de CF, OF, ZF y SF para cada suma.

	Operación	A (bin)	B (bin)	A + B (bin)	CF	OF	ZF	SF
a	60 + 40	?	?	?	?	?	?	?
b	100 + 100	?	?	?	?	?	?	?
c	−1 + 1	?	?	?	?	?	?	?
d	−128 + (−1)	?	?	?	?	?	?	?

Método: 1) pasar a binario de 8 bits (C2 si es negativo); 2) sumar y descartar el bit 9; 3) CF = ¿salió un 1?; OF = ¿mismo signo en A y B y signo distinto en el resultado?

Ejercicio resuelto · Solución

	Operación	A	B	A + B	CF	OF	ZF	SF
a	60 + 40	0011 1100	0010 1000	0110 0100 = 100	0	0	0	0
b	100 + 100	0110 0100	0110 0100	1100 1000 = −56	0	1	0	1
c	−1 + 1	1111 1111	0000 0001	0000 0000 = 0	1	0	1	0
d	−128 + (−1)	1000 0000	1111 1111	0111 1111 = 127	1	1	0	0

b y d
OF = 1: el resultado con signo está mal (−56 y 127).

c
CF = 1 pero OF = 0: sin signo falla (255 + 1), con signo es correcto.

Idea clave
La misma suma, dos lecturas: el programa elige qué bandera revisar.

Reto · Descifra el volcado hexadecimal

datos.bin · xxd

```
00000000: 2c01 0000 feff 41c3 b16f 2032 3032 3600  ,.....A..o 2026.
```

2C	01	00	00	FE	FF	41	C3	B1	6F	20	32	30	32	36	00
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

↑ *desplazamiento de cada byte (desde 0)*

En parejas, responde (máquina little-endian):

1. Entero de 32 bits sin signo en el desplazamiento 0.

2. Entero de 16 bits con signo en el desplazamiento 4.

3. Texto UTF-8 desde el desplazamiento 6 hasta el byte 00. ¿Cuántos caracteres y cuántos bytes?

4. Si una máquina big-endian leyera el mismo entero de 32 bits sin signo de la pregunta 1, ¿qué valor obtendría?

Tabla de apoyo

Hex	Carácter
20	espacio
30 … 39	0 … 9
41	A
61 … 7A	a … z
6F	o
C3 B1	ñ (2 bytes)
00	fin de cadena

Desplazamiento (offset) = número de bytes desde el inicio del archivo. Pueden verificar con el Laboratorio de Memoria y Caché, pestaña Datos.

Reto · Solución

1	`2C 01 00 00`	0x0000012C = 300
2	`FE FF`	0xFFFE = −2 (C2 de 16 bits)
3	`41 C3 B1 6F 20 32 30 32 36 00`	«Año 2026»: 8 caracteres en 9 bytes (+ el 00)
4	`2C 01 00 00`	0x2C010000 = 738 263 040

Moraleja: los bytes no dicen qué son. El mismo volcado es número, texto o basura según el tipo y el orden con que se lea.

Quiz de la sesión 1 · Pregunta 1 de 2

Justo después de ejecutar CALL, ¿qué hay en la cima de la pila?

A) El valor de RAX

B) La dirección de retorno

C) El código de la función llamada

D) Nada: CALL no usa la pila

Fuera del presupuesto · según el tiempo

Quiz de la sesión 1 · Respuesta 1 de 2

Pregunta: ¿qué hay en la cima de la pila justo después de CALL?

A) El valor de RAX

B) **La dirección de retorno** ✓

C) El código de la función llamada

D) Nada: CALL no usa la pila

La nota de «dónde volver»

CALL hace dos cosas: **`PUSH`** de la dirección de la instrucción siguiente y salto a la función.

RET la saca de la cima y la pone en **`RIP`**.

A es un error común: RAX solo va a la pila con un PUSH explícito.

Quiz de la sesión 1 · Pregunta 2 de 2

En 8 bits, el patrón 1111 1111 (0xFF), ¿cuánto vale sin signo y con signo (C2)?

A) 255 y 255

B) 255 y −1

C) −1 y 255

D) 127 y −128

Fuera del presupuesto · según el tiempo

Quiz de la sesión 1 · Respuesta 2 de 2

Pregunta: valor de 1111 1111 sin signo y con signo (8 bits).

A) 255 y 255

B) **255 y −1** ✓

C) −1 y 255

D) 127 y −128

Mismos bits, dos lecturas

Sin signo: 128 + 64 + … + 1 = **255**.

Con signo: invertir → 0000 0000, +1 → 1 → el valor es **−1**.

D son los extremos de C2 en 8 bits: 127 = 0111 1111 y −128 = 1000 0000.

Cierre de la sesión 1 (Puntos clave)

Lo que el cocinero tiene a su alcance

1. **El programa ve solo una parte de la CPU:** registros, banderas, modos y pila.



Lo que el cocinero tiene en la mano.

2. **El modo usuario/núcleo es la base de la protección:** instrucciones privilegiadas solo en anillo 0.



Zonas donde solo entra el administrador.

3. **La pila retoma lo interrumpido en orden inverso:** CALL guarda la dirección de retorno y RET la recupera.



Las notas de «dónde me quedé».

4. **Los bits no traen tipo:** complemento a 2, banderas y endianness les dan significado.



0x9C puede ser 156 o −100.

Próxima sesión: ¿por qué el cocinero pasa tanto tiempo esperando? Jerarquía de memoria, localidad y caché. Traer gcc instalado (Linux o WSL) para la demo.



Sesión 2 · Jerarquía de memoria y caché

Tema 1.5 · brecha CPU–memoria, localidad, caché y tiempo medio de acceso · 1 h 45 min

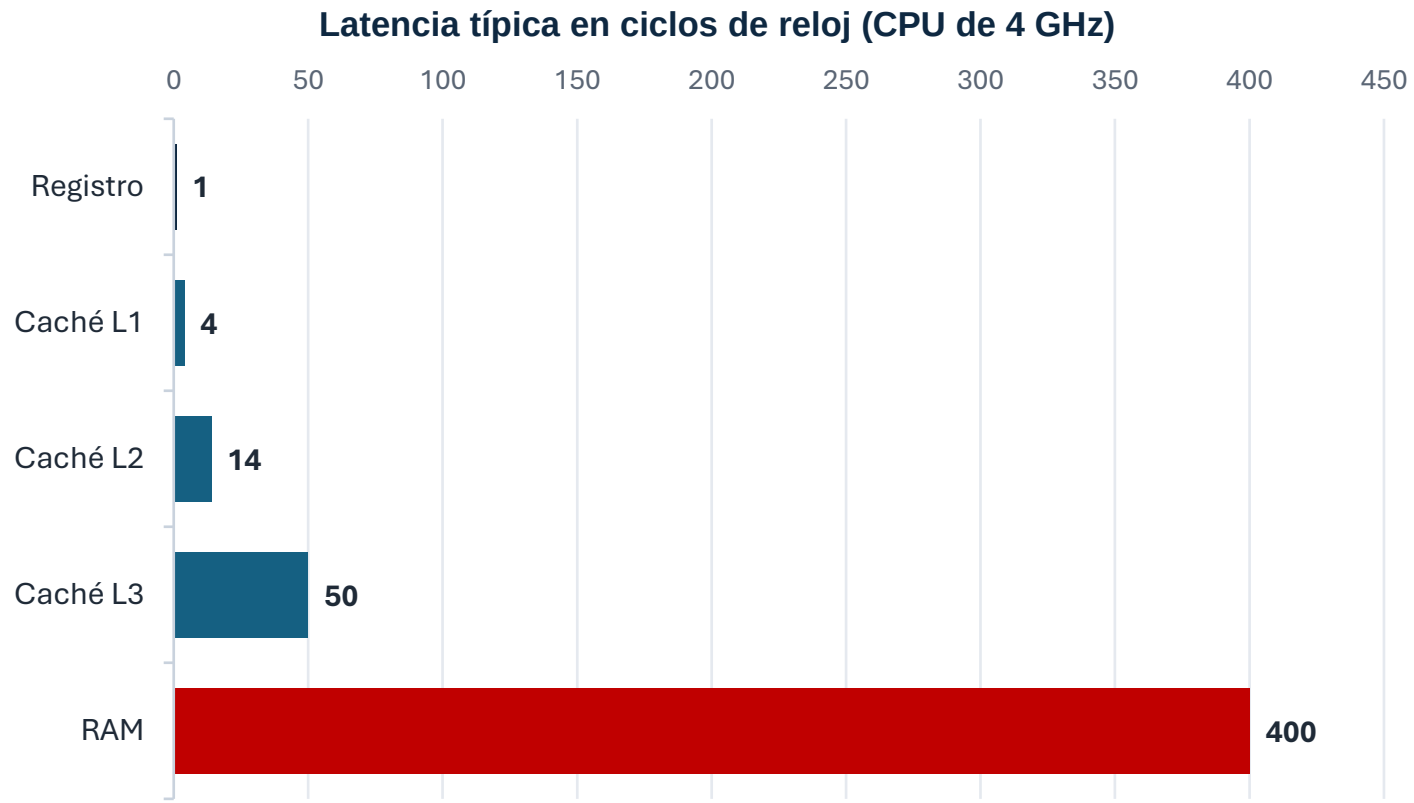
La brecha entre la CPU y la memoria

¿Por qué el cocinero espera tanto?

0,25 ns
un ciclo de una CPU de 4 GHz

≈ 100 ns
una lectura de la RAM (DRAM)

≈ 400 ciclos de espera por cada dato que no esté en caché.



Valores aproximados y redondeados · Hennessy y Patterson (2019), Computer Architecture, cap. 2 · Drepper, U. (2007), What Every Programmer Should Know About Memory.

La jerarquía de memoria

Mano, estante, mesada, bodega, proveedor

Mano

Lo que usa ahora. Casi nada cabe.

Estante

Junto al fogón: lo de esta receta.

Mesada

Todo lo del día. Hay que cruzar la cocina.

Bodega

Mucho espacio; bajar toma tiempo.

Proveedor

Sin límite… pero llega tarde.

De la cocina a la CPU

En la cocina	En la computadora
Mano	**Registros · < 1 KB · 0,25 ns**
Estantes junto al fogón	**Cachés L1–L3 · KB a MB · 1–12 ns**
Mesada	**RAM · GB · ≈ 100 ns**
Bodega	**SSD / disco · TB · 0,1–10 ms**
Proveedor	**Red / nube · ms a s**

Idea clave

Cada nivel es **más grande, más barato y más lento** que el de arriba, y guarda una copia de lo que se está usando.

Quién lo gestiona: compilador, hardware y, entre mesada y bodega, el SO.

Valores típicos de equipos de escritorio y portátiles · Stallings (2006), OAC, sec. 4.1 · Tanenbaum y Bos (2022), sec. 1.3.2.

Si un ciclo de la CPU durara 1 segundo...

Si tomar algo de la mano tardara un segundo...

Acceso	Tiempo real	Escala humana
Registro	0,25 ns	1 s
Caché L1	1 ns	4 s
Caché L2	≈ 4 ns	16 s
Caché L3	≈ 12 ns	48 s
RAM	≈ 100 ns	≈ 7 minutos
SSD NVMe	≈ 100 µs	≈ 4,6 días
Disco duro (HDD)	≈ 5 ms	≈ 8 meses
Paquete Quito ↔ Madrid	≈ 150 ms	≈ 19 años

Lectura

Un fallo de caché que va a la RAM cuesta lo que una pausa de 7 minutos en una conversación.

Ir al disco es esperar días o meses: por eso el SO hace todo lo posible por evitarlo.

Adaptado de Gregg, B. (2020), Systems Performance (2.ª ed.), tabla 2.2 · Scott, C., «Latency Numbers Every Programmer Should Know» (versión interactiva).

El principio de localidad

La sal a mano y la caja entera de tomates

1 · La sal, a mano

La usó hace un momento y la volverá a usar. No la guarda: la deja cerca.

2 · La caja entera

Si necesita un tomate, pronto necesitará el de al lado. Trae la caja, no un tomate.

3 · En un bucle

suma.c

```
for (i = 0; i < n; i++)
  suma += a[i];
```

De la cocina a la CPU

En la cocina	En la computadora
Volver a usar la sal	**Localidad temporal (suma, i)**
Usar el tomate de al lado	**Localidad espacial (a[i], a[i+1])**
Traer la caja entera	**Línea de caché: 64 B = 16 enteros**
Dejar a mano lo reciente	**La caché guarda lo reciente**

Idea clave

Los programas no usan la memoria al azar: **repiten y usan vecinos**.

La caché apuesta a eso; si el programa no lo hace, la apuesta se pierde.

Denning, P. J. (2005), «The Locality Principle», Communications of the ACM 48(7) · Stallings (2006), OAC, sec. 4.1.

Demo: la matriz por filas o por columnas

El cocinero que cruza la cocina por cada tomate

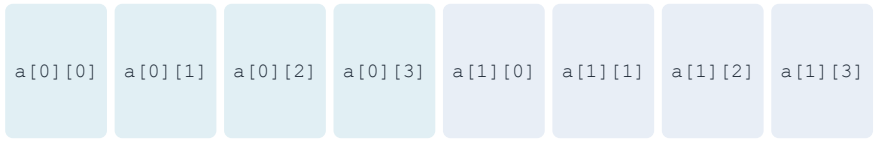
matriz.c (fragmento)

```
#define N 4096
static int a[N][N];          // 64 MiB

for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        s += por_columnas ? a[j][i]
                          : a[i][j];

$ gcc -O0 matriz.c -o matriz
$ ./matriz           # por filas
filas     0.047 s
$ ./matriz c         # por columnas
columnas  0.130 s
```

C guarda la matriz fila por fila



memoria contigua →

Por filas
salto de 4 B: usa toda la caja (16 enteros por línea)

Por columnas
salto de 16 KiB: un tomate de cada caja; cruza cada vez

Mismo trabajo, mismas instrucciones, 2 a 10 veces más lento según el equipo.

Código completo en las notas del docente · Ostrovsky, I. (2010), «Gallery of Processor Cache Effects» · Drepper (2007), sec. 6.2.

La caché: líneas, aciertos y fallos

¿Está en el estante o hay que cruzar la cocina?

1 · Mira el estante

Necesita un tomate. Primero mira el estante junto al fogón.

2 · ¡Está!

Lo toma al instante y sigue cocinando.

3 · No está

Cruza hasta la mesada y trae la caja entera. Si no hay sitio, devuelve otra.

4 · Lleva la cuenta

De cada 100 veces, ¿cuántas lo encontró en el estante?

De la cocina a la CPU

En la cocina	En la computadora
El estante junto al fogón	**Caché**
La caja de tomates	**Línea o bloque (64 B)**
Encontrarlo en el estante	**Acierto (hit) · ≈ 1 ns**
Cruzar hasta la mesada	**Fallo (miss) · ≈ 100 ns, trae la línea**
«De cada 100, cuántas»	**Tasa de aciertos h = aciertos / accesos**

Idea clave

La CPU pide palabras sueltas; la caché trae **líneas completas** de la RAM.

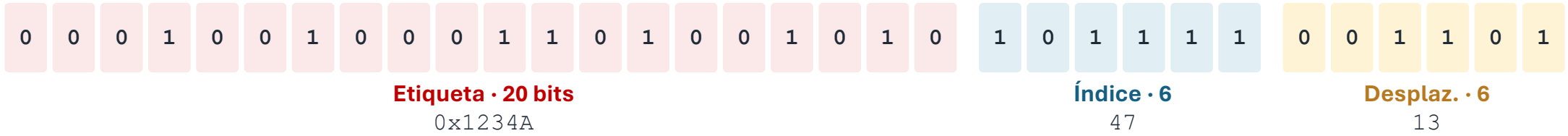
h resume si un programa «se lleva bien» con la caché.

Stallings (2006), OAC, sec. 4.2 «Elementos de diseño de la caché» · Hennessy y Patterson (2019), apéndice B.

Etiqueta, índice y desplazamiento

Repisa, frasco y posición dentro de la caja

Ejemplo: C = 32 KiB, L = 64 B, A = 8 vías → S = C / (L·A) = 64 conjuntos



Desplazamiento

posición dentro de la caja

Qué byte dentro de la línea: $\log_2 L = \log_2 64 = 6$ bits.

Índice

en qué repisa mirar

En qué conjunto buscar: $\log_2 S = \log_2 64 = 6$ bits.

Etiqueta

¿es esta la caja que busco?

El resto (32 − 6 − 6 = 20 bits): se compara con las 8 vías del conjunto.

Acierto ⇔ alguna vía válida del conjunto 47 tiene la etiqueta 0x1234A.

Bryant y O'Hallaron (2016), CS:APP, sec. 6.4 «Cache Memories» · Stallings (2006), OAC, sec. 4.3.

Dónde se puede guardar cada línea

Puesto fijo, repisa asignada o donde haya espacio

Correspondencia directa

`A = 1`



Puesto fijo: cada caja tiene un único lugar.

+ simple y rápida

− conflictos: dos bloques que caen en el mismo lugar se expulsan entre sí

Asociativa por conjuntos

`A = 2, 4, 8…`



Repisa asignada: cualquiera de sus A lugares.

+ menos conflictos

− hay que comparar A etiquetas a la vez

Totalmente asociativa

`un solo conjunto`



Donde haya espacio: cualquier línea.

+ sin conflictos

− costosa: compara todas las etiquetas

Si el conjunto está lleno: LRU expulsa la línea usada hace más tiempo; FIFO, la que entró primero; aleatorio, cualquiera.

Stallings (2006), OAC, sec. 4.3 «Funciones de correspondencia» y «Algoritmos de sustitución».

Tiempo medio de acceso

Lo que cuesta, en promedio, conseguir un ingrediente

$$T_{medio} = T_c + (1 - h) \cdot T_m$$

siempre se consulta la caché; solo los fallos pagan además la RAM

h	T medio (Tc = 1 ns, Tm = 100 ns)	Peso de los fallos
0,80	$1 + 0{,}20 \cdot 100 = 21$ ns	95 %
0,90	$1 + 0{,}10 \cdot 100 = 11$ ns	91 %
0,95	$1 + 0{,}05 \cdot 100 = 6$ ns	83 %
0,99	$1 + 0{,}01 \cdot 100 = 2$ ns	50 %

Dos niveles (L1 + L2)

$$T = T_1 + (1 - h_1) \cdot [\, T_2 + (1 - h_2) \cdot T_m \,]$$

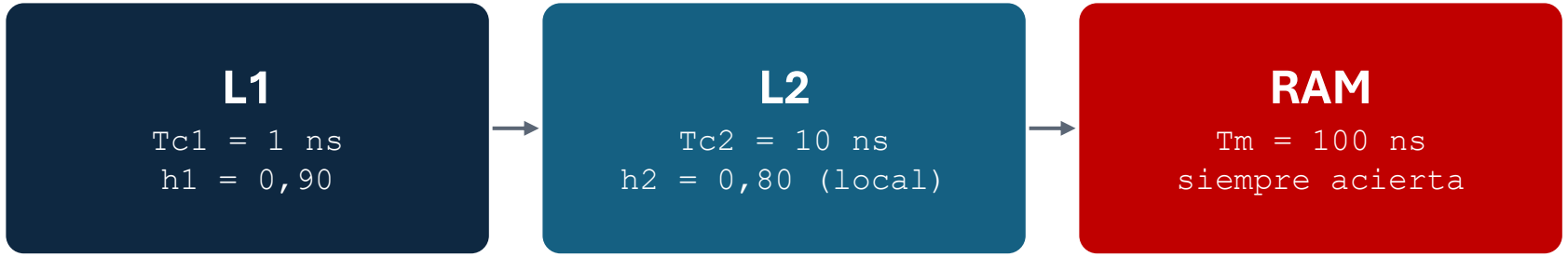
h_2 es la tasa de aciertos de L2 sobre los accesos que fallaron en L1 (tasa local).

Intuición: subir h de 95 % a 99 % divide el tiempo por 3. Los fallos dominan el costo; por eso importa tanto la localidad.

Hennessy y Patterson (2019), apéndice B.2, ecuación del AMAT · Ya presentada en la semana 1 (tema 1.3, métricas).

Ejercicio resuelto · ¿Vale la pena una caché L2?

¿Vale la pena un segundo estante?



a) Tiempo medio si la CPU solo tuviera L1.

b) Tiempo medio con L1 y L2.

c) Sin L2, ¿qué h1 haría falta para igualar el resultado de b)?

Fórmulas

1 nivel:

$Tc + (1-h) \cdot Tm$

2 niveles:

$T1 + (1-h1) \cdot [T2 + (1-h2) \cdot Tm]$

Antes de calcular: estimen a ojo cuánto mejora la L2. ¿El doble? ¿Diez veces?

Ejercicio resuelto · Solución

a) **Solo L1** $T = 1 + 0{,}10 \cdot 100$ **11 ns**

b) **L1 + L2** $T = 1 + 0{,}10 \cdot (10 + 0{,}20 \cdot 100) = 1 + 0{,}10 \cdot 30$ **4 ns**

c) **h1 necesaria sin L2** $1 + (1 - h1) \cdot 100 = 4 \rightarrow 1 - h1 = 0{,}03$ **h1 = 0,97**

Conclusión: la L2 hace el acceso medio 2,75 veces más rápido (11 → 4 ns). Lograr lo mismo solo con L1 exigiría pasar de 90 % a 97 % de aciertos, que es mucho más difícil.

Laboratorio · Experimentos con el simulador

#	Escenario en el laboratorio	Cambio a probar	Anota h
1	A · Matriz por filas	—	____ %
2	B · Matriz por columnas	Sube C hasta que h ≈ 94 %	____ % · C = ____
3	D · Conflicto en directa	Asociatividad 1 → 2 vías	____ % → ____ %
4	C · Stride de 64 B	Línea L = 64 B → 128 B	____ % → ____ %
5	E · Filtro de imagen	Programa: por columnas → por filas	____ % → ____ %

Laboratorio de Memoria y Caché

Pestaña 1 · Caché

Usen «Ejecutar todo» y lean h y T medio en el panel superior.

Para pensar: ¿en qué experimentos cambió el hardware y en cuáles cambió el programa? ¿Cuál de las dos cosas puedes controlar tú como programador?

Laboratorio de Memoria y Caché: claude.ai/artifact/XHwb9LMrWHc8KeEUnTmV7g · La guía del estudiante amplía estos experimentos con seis escenarios.

Reto · ¿Qué tasa de aciertos necesitamos?

Un sistema embebido tiene una caché con **`Tc = 2 ns`** y una RAM con **`Tm = 120 ns`**. Se exige un tiempo medio de acceso de **8 ns o menos**.

1. ¿Qué tasa de aciertos h mínima necesita el programa?

2. Si el programa solo logra h = 0,90, ¿qué Tm máximo toleraría la RAM?

3. ¿Cumpliría el requisito el recorrido por columnas del escenario B (h = 0)? ¿Cuál sería su T medio?



Pista

Despeja (1 − h) de la ecuación:

$Tc + (1 - h) \cdot Tm \leq 8$

Comprueben su respuesta en el laboratorio moviendo Tc y Tm.

Reto · Solución

1 $2 + (1 - h) \cdot 120 \leq 8 \rightarrow 1 - h \leq 0{,}05$ **$h \geq 0{,}95$**

2 $2 + 0{,}10 \cdot Tm \leq 8 \rightarrow Tm \leq 60$ **$Tm \leq 60$ ns**

3 $2 + (1 - 0) \cdot 120$ **122 ns ✗**

Lectura: el requisito depende más del programa que de la caché. Con un recorrido sin localidad (h = 0) el tiempo medio es 15 veces el permitido, aunque el hardware sea el mismo.

Su equipo por dentro: comandos

¿Qué tan grandes son los estantes y la mesada de tu equipo?

bash · Linux

```
$ lscpu --caches
NAME ONE-SIZE ALL-SIZE WAYS TYPE        LEVEL
L1d       48K     384K   12 Data            1
L1i       32K     256K    8 Instruction     1
L2      1.3M      10M   10 Unified         2
L3        24M      24M   12 Unified         3

$ getconf LEVEL1_DCACHE_LINESIZE
64

$ free -h
               total        used        free      ...   available
Mem:            15Gi      4.1Gi      7.9Gi      ...        11Gi
Swap:          4.0Gi          0B      4.0Gi
```

PowerShell · Windows

```
PS> Get-CimInstance Win32_CacheMemory |
      Select-Object Purpose, InstalledSize

Purpose  InstalledSize   # en KB
L1 Cache           640
L2 Cache         10240
L3 Cache         24576
```

Calcula: con los datos de L1d de la izquierda (48 KiB, 12 vías, líneas de 64 B), ¿cuántos conjuntos tiene? ¿Cuántos bits de índice?

lscpu (util-linux) · free (procps) · Win32_CacheMemory (CIM/WMI) · Salidas de ejemplo; varían según el equipo.

Quiz de la sesión 2 · Pregunta 1 de 2

Ante un fallo, la caché trae 64 bytes en lugar de solo el dato pedido. ¿Qué aprovecha?

A) Localidad temporal

B) Localidad espacial

C) La asociatividad

D) El reemplazo LRU

Fuera del presupuesto · según el tiempo

Quiz de la sesión 2 · Respuesta 1 de 2

Pregunta: ¿qué aprovecha la caché al traer 64 B ante un fallo?

A) Localidad temporal

B) **Localidad espacial** ✓

C) La asociatividad

D) El reemplazo LRU

La caja entera de tomates

Los datos vecinos se usarán pronto: eso es **localidad espacial**.

La temporal se aprovecha guardando lo usado hace poco, no trayendo vecinos.

C y D son decisiones de diseño (dónde y qué expulsar), no la razón de traer 64 B.

Quiz de la sesión 2 · Pregunta 2 de 2

Caché de 32 KiB, líneas de 64 B y 8 vías: ¿cuántos conjuntos tiene?

A) 8

B) 64

C) 512

D) 4096

Fuera del presupuesto · según el tiempo

Quiz de la sesión 2 · Respuesta 2 de 2

Pregunta: conjuntos de una caché de 32 KiB, L = 64 B, A = 8.

A) 8

B) **64** ✓

C) 512

D) 4096

S = C / (L·A)

S = 32 768 / (64 · 8) = 32 768 / 512 = **64**.

Son 6 bits de índice y 6 de desplazamiento.

C (512) es el número de líneas: falta dividir entre las 8 vías.

Cierre de la sesión 2 (Puntos clave)

Por qué el cocinero espera, y cómo lo evita

1. **La memoria es lenta frente a la CPU:** ir a la RAM cuesta ≈ 400 ciclos. — *Cruzar la cocina por cada ingrediente.*
2. **La jerarquía y la localidad lo disimulan:** cada nivel guarda copia de lo que se usa. — *Mano, estante, mesada, bodega.*
3. **La caché usa los bits de la dirección:** etiqueta, índice y desplazamiento; trae líneas completas. — *Repisa, frasco y caja entera.*
4. **T = Tc + (1 − h)·Tm:** los fallos dominan; el orden de los bucles puede valer más que el hardware. — *Usar toda la caja antes de cruzar.*

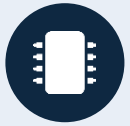
Próxima sesión (1 h): quiz de la semana, el caso del cocinero que cruza la cocina un millón de veces y el Deber 2.

Sesión 3 · Consolidación

Quiz, caso integrador y Deber 2 · 50 minutos efectivos

Mapa de la semana

La semana en el restaurante



Modelo de programación

registros · ISA · modos · pila

lo que el cocinero tiene a mano y las zonas con llave



Representación de datos

C2 · banderas · endianness · UTF-8 · IEEE 754

los bits no traen tipo: el programa decide cómo leerlos



Jerarquía de memoria

localidad · caché · h · T medio

mano, estante, mesada y bodega

las instrucciones operan sobre bits de n bits y actualizan banderas

esos bits viven en memoria, ordenados byte a byte

la velocidad depende de dónde están: registro, caché o RAM

En el restaurante: ya sabemos qué tiene el cocinero a mano, qué zonas no puede pisar y por qué espera ingredientes. Falta conocer al que manda: el administrador, es decir, el SO (semana 3).

Quiz de la semana · Pregunta 1 de 5

Un programa en modo usuario ejecuta la instrucción HLT (detener la CPU). ¿Qué ocurre?

A) La CPU se detiene hasta la siguiente interrupción

B) La instrucción se ignora sin efecto

C) La CPU genera una excepción y el SO toma el control

D) El programa pasa a modo núcleo automáticamente

Quiz de la semana · Respuesta 1 de 5

Pregunta: HLT en modo usuario

A) La CPU se detiene hasta la siguiente interrupción

B) La instrucción se ignora sin efecto

C) La CPU genera una excepción y el SO toma el control

D) El programa pasa a modo núcleo automáticamente

¿Por qué?

HLT es una instrucción privilegiada: solo se permite en anillo 0.

La CPU no la ejecuta: lanza una excepción de protección general y salta a la rutina del SO.

En Linux el proceso termina con la señal SIGSEGV.

Quiz de la semana · Pregunta 2 de 5

En 8 bits con signo se suma 100 + 50. ¿Qué resultado queda en el registro y qué bandera se activa?

A) 150, ninguna bandera

B) −106, OF = 1

C) 150, CF = 1

D) −106, CF = 1

Quiz de la semana · Respuesta 2 de 5

Pregunta: 100 + 50 en 8 bits con signo

A) 150, ninguna bandera

B) **−106, OF = 1** ✓

C) 150, CF = 1

D) −106, CF = 1

¿Por qué?

```
0110 0100 + 0011 0010 = 1001 0110 = 0x96
```

Con signo, 0x96 = −106: dos positivos dieron un negativo → OF = 1.

Sin signo vale 150 ≤ 255, así que CF = 0.

Quiz de la semana · Pregunta 3 de 5

El entero 0x12345678 se guarda a partir de la dirección 0x1000 en un equipo x86-64. ¿Qué byte hay en 0x1000?

A) 0x12

B) 0x78

C) 0x21

D) 0x87

Quiz de la semana · Respuesta 3 de 5

Pregunta: 0x12345678 en 0x1000, x86-64

A) 0x12

B) **0x78**

C) 0x21

D) 0x87

¿Por qué?

x86-64 es little-endian: el byte menos significativo va en la dirección más baja.

`0x1000: 78 · 0x1001: 56 · 0x1002: 34 · 0x1003: 12`

C y D invierten dígitos dentro del byte: el orden de bytes nunca cambia los bits de cada byte.

Quiz de la semana · Pregunta 4 de 5

En C, ¿por qué sumar una matriz grande recorriéndola por columnas es más lento que por filas?

A) Porque se ejecutan más instrucciones

B) Porque cada acceso cae en una línea distinta y se pierde la localidad espacial

C) Porque las columnas se guardan en disco

D) Porque el compilador no puede optimizar bucles anidados

Quiz de la semana · Respuesta 4 de 5

Pregunta: matriz por columnas en C

A) Porque se ejecutan más instrucciones

B) Porque cada acceso cae en una línea distinta y se pierde la localidad espacial

C) Porque las columnas se guardan en disco

D) Porque el compilador no puede optimizar bucles anidados

¿Por qué?

C guarda las matrices fila por fila. Por columnas, dos accesos seguidos están a N·4 bytes de distancia.

Cada acceso trae una línea de 64 B de la que solo se usan 4 bytes: casi todo son fallos.

El número de instrucciones es el mismo.

Quiz de la semana · Pregunta 5 de 5

Una caché tiene Tc = 1 ns, la RAM Tm = 100 ns y el programa logra h = 0,95. ¿Cuál es el tiempo medio de acceso?

A) 1,95 ns

B) 5 ns

C) 6 ns

D) 95 ns

Quiz de la semana · Respuesta 5 de 5

Pregunta: Tc = 1 ns, Tm = 100 ns, h = 0,95

A) 1,95 ns

B) 5 ns

C) **6 ns** ✓

D) 95 ns

¿Por qué?

$$T = Tc + (1 - h) \cdot Tm$$

$$\mathbf{T = 1 + 0{,}05 \cdot 100 = 6\ ns}$$

B olvida sumar Tc. D usa h en lugar de (1 − h).

Caso · El filtro de imagen lento

El cocinero que cruza la cocina un millón de veces

Situación

Una app aplica un desenfoque vertical a fotos de 1024 × 1024 píxeles (4 bytes por píxel, RGBA). Tarda mucho más de lo esperado en un equipo con una L1 de 32 KiB, líneas de 64 B y 8 vías.

filtro.c

```
#define W 1024
#define H 1024
uint32_t img[H][W], out[H][W];

for (int x = 0; x < W; x++)        // columnas
    for (int y = 1; y < H - 1; y++)
        out[y][x] = prom(img[y-1][x],
                         img[y][x], img[y+1][x]);
```

En parejas, predigan antes de simular

1. ¿Cuánto ocupa la imagen? ¿Cabe en la L1?
2. ¿Cuántos bytes separan img[y][x] de img[y+1][x]?
3. Predigan h para el recorrido actual.
4. ¿Qué cambio de una línea lo arregla?

Caso · Lo comprobamos en el simulador

Contemos los viajes hasta la mesada

1. **Cargar el escenario E · Filtro de imagen**
   N = 1024, C = 32 KiB, L = 64 B, 8 vías
2. **Pulsar «Ejecutar todo»**
   1 048 576 accesos
3. **Cambiar el programa a «por filas»**
   mismo hardware
4. **Mover C en la gráfica**
   ¿desde qué tamaño ayuda la caché?

Por columnas

h

0 %

T medio

101 ns

Por filas

h

93,8 %

T medio

7,25 ns

Resultado: intercambiar dos líneas de código hace el acceso medio unas 14 veces más rápido (101 → 7,25 ns), sin cambiar el hardware.

Laboratorio de Memoria y Caché, escenario E · https://claude.ai/artifact/XHwb9LMrWHc8KeEUnTmV7g

Caso · Y el píxel, ¿cuánto vale?

Se quiere revisar el píxel (x = 10, y = 3) directamente en el archivo binario de la imagen.

```
img.raw · xxd -s 0x3028 -l 4
00003028: 4080 c0ff                        @...
```

1. ¿Por qué el volcado empieza en el desplazamiento 0x3028?

2. Leído como uint32_t en x86-64, ¿qué valor tiene el píxel?

3. Si los bytes son R, G, B, A en ese orden, ¿cuánto vale cada canal en decimal?

Respuestas

1. `(3 · 1024 + 10) · 4 = 12 328 = 0x3028`

2. `little-endian → 0xFFC08040`

3. `R = 64, G = 128, B = 192, A = 255`

Un azul claro opaco.

Deber 2 · Reporte de memoria del equipo

El reporte debe incluir

- Nombre del equipo, SO y fecha
- Arquitectura (x86_64 / ARM64) y orden de bytes
- Cachés L1d, L1i, L2 y L3: tamaño, vías y línea (lo que el SO reporte)
- Número de conjuntos de la L1d: S = C / (L · A)
- RAM total y libre; swap o archivo de paginación
- Discos: nombre, tipo (SSD/HDD) y tamaño
- Guardar todo en memoria_<equipo>.txt

Extra: añadir al reporte los tiempos de ./matriz y ./matriz c medidos en ese equipo.

Modalidad
En parejas (las mismas del portafolio)

Entrega
GitHub · carpeta semana02/ · enlace en el aula virtual

Fecha
[completar]

Rúbrica (25 % c/u)
Funciona en ambos SO · documentado · organizado · commits de ambos

Deber 2 · Esqueleto para empezar

memoria.sh · Bash

```bash
#!/usr/bin/env bash
# memoria.sh — Deber 2, semana 2
salida="memoria_$(hostname).txt"
{
  echo "== Equipo: $(hostname) $(date +%F)"
  echo "== Arquitectura: $(uname -m)"
  lscpu | grep 'Byte Order'
  echo "== Cachés"; lscpu --caches
  echo "== RAM y swap"; free -h
  # TODO: discos -> lsblk -d -o NAME,ROTA,SIZE
  # TODO: conjuntos L1d = C / (L * A)
} > "$salida"
echo "Reporte: $salida"
```

memoria.ps1 · PowerShell

```powershell
# memoria.ps1 — Deber 2, semana 2
$salida = "memoria_$env:COMPUTERNAME.txt"
& {
  "== Equipo: $env:COMPUTERNAME $(Get-Date -f d)"
  "== Arquitectura: $env:PROCESSOR_ARCHITECTURE"
  "LittleEndian: $([BitConverter]::IsLittleEndian)"
  "== Cachés (KB)"
  Get-CimInstance Win32_CacheMemory |
    Select-Object Purpose, InstalledSize
  # TODO: RAM -> Win32_OperatingSystem
  # TODO: discos -> Get-PhysicalDisk
} | Out-File $salida
"Reporte: $salida"
```

Temas para revisión autónoma

IEEE 754 en detalle	Sesgo, números normalizados y desnormalizados, ±∞ y NaN.	*CS:APP, sec. 2.4 · Goldberg (1991)*
Codificación UTF-8	Cómo se codifican caracteres de 1 a 4 bytes; diferencia con ASCII y UTF-16.	*RFC 3629, sec. 3 · CS:APP, sec. 2.1.4*
Modos de direccionamiento	Inmediato, directo, indirecto, por registro, con desplazamiento.	*Stallings (2006), OAC, cap. 11*
Convenciones de llamada	System V AMD64 (Linux) frente a Windows x64: registros de argumentos.	*CS:APP, sec. 3.7 · Microsoft Learn, «x64 calling convention»*
Políticas de escritura de la caché	Write-through frente a write-back; coherencia entre núcleos.	*Stallings (2006), OAC, sec. 4.3*

Las llamadas al sistema (strace, POSIX, Win32) no son tarea: se verán en clase en la semana 4.

Conclusiones de la semana

Modelo de programación

Registros, ISA, modos y pila. Los modos usuario/núcleo son la base de la protección del SO.

Datos

Los bits no tienen tipo: complemento a 2, banderas, endianness y UTF-8 les dan significado.

Memoria

Jerarquía + localidad + caché. T medio = Tc + (1 − h)·Tm: el programa decide buena parte de h.

Semana 3 · ¿Quién manda en la cocina? 2.1 Tipos de SO · 2.2 Componentes del SO · 2.3 Arranque y parada. Llega el administrador: el sistema operativo por dentro.

Anexo · Ejercicios adicionales

Opcionales · para estudio autónomo o para usar el margen de la sesión 3

Ejercicio adicional 1 · Complemento a 2

a) Representa −37 en 8 bits con complemento a 2. Escribe el resultado en binario y en hexadecimal.

b) El registro AL contiene 0xC8. ¿Qué valor tiene si se interpreta sin signo? ¿Y con signo?

c) ¿Cuál es el menor y el mayor entero con signo que cabe en 16 bits?

Herramienta: comprueba tus respuestas en el Laboratorio de Memoria y Caché, pestaña Datos.

Ejercicio adicional 1 · Solución

a) `37 = 0010 0101 → invertir 1101 1010 → + 1` **1101 1011 = 0xDB**

b) `0xC8 = 1100 1000` **200 sin signo · −56 con signo**

c) $-2^{15} \ldots 2^{15} - 1$ **−32 768 … 32 767**

Comprobación de a): 0xDB + 37 = 0xDB + 0x25 = 0x100 → en 8 bits queda 0x00. Un número más su opuesto da cero.

Ejercicio adicional 2 · Campos de una dirección

Caché de **16 KiB**, líneas de **32 B**, asociativa de **4 vías**. Direcciones de **32 bits**.

a) ¿Cuántos conjuntos tiene la caché?

b) ¿Cuántos bits ocupan el desplazamiento, el índice y la etiqueta?

c) ¿En qué conjunto se busca la dirección **`0x00403A64`**? ¿Cuál es su etiqueta?

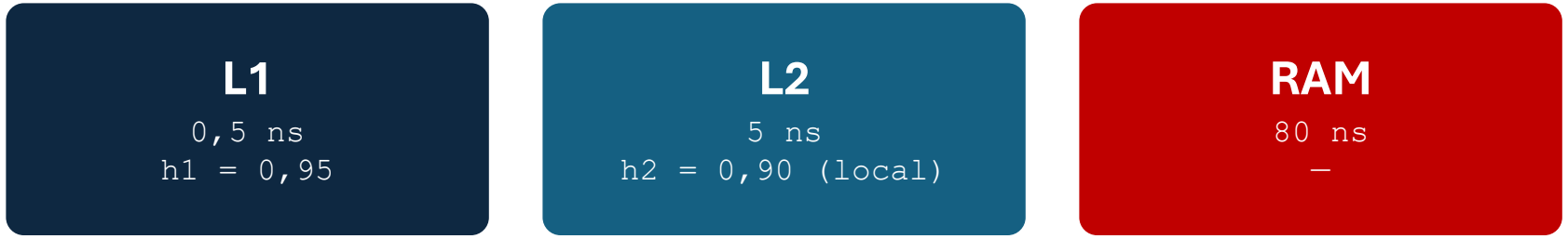
Pista: escribe solo los 12 bits bajos de la dirección (0xA64) en binario; el resto es etiqueta.

Ejercicio adicional 2 · Solución

Paso	Cálculo	Resultado
a) Conjuntos	$S = 16\,384 / (32 \cdot 4)$	**128**
b) Desplazamiento	$\log_2 32$	**5 bits**
b) Índice	$\log_2 128$	**7 bits**
b) Etiqueta	$32 - 7 - 5$	**20 bits**
c) Bits bajos de 0xA64	$1010011 \mid 00100$	**índice 83 · desplaz. 4**
c) Etiqueta	$0x00403A64 >> 12$	**0x00403**

Verifica: en el laboratorio, C = 16 KiB, L = 32 B, A = 4 da «512 líneas… en 128 conjuntos de 4 vías».

Ejercicio adicional 3 · Dos niveles de caché



a) Calcula el tiempo medio de acceso con L1 y L2.

b) Calcula el tiempo medio si se elimina la L2.

c) ¿Qué porcentaje de todos los accesos llega a la RAM con L2?

Ejercicio adicional 3 · Solución

a) $0{,}5 + 0{,}05 \cdot (5 + 0{,}10 \cdot 80) = 0{,}5 + 0{,}05 \cdot 13$ **1,15 ns**

b) $0{,}5 + 0{,}05 \cdot 80$ **4,5 ns**

c) $(1 - h1) \cdot (1 - h2) = 0{,}05 \cdot 0{,}10$ **0,5 %**

Lectura: solo 1 de cada 200 accesos llega a la RAM, y aun así sin L2 el tiempo medio sería casi 4 veces mayor.

Bibliografía y recursos de interés

Bibliografía

- Stallings, W. (2006). *Organización y arquitectura de computadores* (7.ª ed.). Pearson Prentice Hall. Caps. 4, 9–13.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson. Cap. 1.
- Carretero, J., García, F., de Miguel, P. y Pérez, F. (2007). *Sistemas operativos: una visión aplicada* (2.ª ed.). McGraw-Hill. Cap. 1.
- Tanenbaum, A. S. y Bos, H. (2022). *Modern Operating Systems* (5th ed.). Pearson. Sec. 1.3.
- Bryant, R. E. y O'Hallaron, D. R. (2016). *Computer Systems: A Programmer's Perspective* (3rd ed.). Pearson. Caps. 2, 3 y 6.
- Hennessy, J. L. y Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann. Cap. 2, apéndice B.
- Gregg, B. (2020). *Systems Performance* (2nd ed.). Addison-Wesley. Cap. 2.

Recursos en línea

- **Laboratorio de Memoria y Caché (del curso)**
  claude.ai/artifact/XHwb9LMrWHc8KeEUnTmV7g
- Compiler Explorer
  godbolt.org
- Cache Simulator · Univ. de Michigan (EECS 370)
  vhosts.eecs.umich.edu/370simulators/cache/s...
- IEEE-754 Floating Point Converter
  www.h-schmidt.net/FloatConverter/IEEE754.html
- Exposing Floating Point · B. Ciechanowski
  ciechanow.ski/exposing-floating-point/
- Latency Numbers Every Programmer Should Know
  colin-scott.github.io/personal_website/rese...
- What Every Programmer Should Know About Memory · U. Drepper
  people.freebsd.org/~lstewart/articles/cpume...
- Gallery of Processor Cache Effects · I. Ostrovsky
  igoro.com/archive/gallery-of-processor-cach...